Evolving Dataset-Specific Split Scores and Out-of-Bag Residual Correction for Tree Ensembles

Vu Gia Nam, Pham Vu Tuan Dat, Long Doan, Huynh Thi Thanh Binh

Abstract

Tree ensembles are strong tabular learners, yet their predictive behavior is largely determined by fixed, hand-designed split criteria such as Gini impurity or variance reduction, which impose a uniform inductive bias across heterogeneous datasets. We propose EvoGraft, a framework that evolves split scores and grafts out-of-bag residual corrections onto the resulting forest, improving tabular prediction at two complementary levels: tree construction and forest-level prediction. EvoGraft first evolves executable, dataset-specific split-scoring programs, evaluated by predictive fitness on held-out data, so that tree structures adapt to each dataset rather than to a universal impurity measure. After the forest is built, out-of-bag instances are grouped into buckets according to ensemble disagreement and leaf support; within each bucket, cross-validation on out-of-bag residuals selects the correction structure. The parameters of the selected correctors are fitted from out-of-bag residuals, and a disjoint calibration fold sets the strength of the correction. Experiments on fourteen tabular benchmarks spanning seven classification and seven regression tasks show that EvoGraft is competitive with tree-based and LLM-based baselines, obtaining the best mean rank on both classification and regression.

Keywords

Tree Ensembles, Evolutionary Computation, Large Language Models, Tabular Data

1 Introduction

Tabular data are common in many real-world applications, including healthcare [32], finance [9, 24], and manufacturing [6]. Although neural models for tabular learning have improved substantially [14], tree ensembles remain highly competitive on many tabular problems [15]. However, widely used methods still rely on fixed, hand-designed split criteria, such as Gini impurity or variance reduction, which apply the same local objective across datasets [4, 8, 20]. These criteria encode general-purpose preferences for partitioning data, while individual datasets differ in their feature and target distributions, noise levels, and feature–target relationships. The purpose of evolving a split-scoring program is therefore to adapt tree construction to the characteristics of a particular dataset, using predictive performance on development-validation data to guide the search. Prior work has tried to reduce this limitation through evolutionary algorithm design: HEAD-DT evolves complete decision-tree induction algorithms by selecting and combining their components [2], while ESC-GE uses grammatical evolution to generate split criteria from a predefined grammar [3]. These studies suggest that evolutionary search can replace fixed, hand-designed rules with data-adaptive tree construction, motivating our focus on evolving dataset-specific split-scoring programs for tree ensembles.

The emergence of large language models (LLMs) has introduced a new direction for machine learning, where LLMs are used not only as predictors but also to design and improve components of the learning pipeline. For tabular data, LLMs have been used as predictors [11, 17, 39], as feature engineers [16, 18], and to guide decision-tree induction [26, 36], while DeLTa [37] uses an LLM-generated rule to correct the errors of an existing tree-based predictor. These approaches motivate studying how dataset-specific tree construction and forest-level residual correction can be combined within a single framework.

To address this gap, we propose EvoGraft, a framework that adapts both tree construction and forest-level correction to each dataset. EvoGraft first evolves, with an LLM as the variation operator, an executable split-scoring program that is used throughout tree construction instead of relying on a fixed criterion. After the ensemble is built, it groups out-of-bag (OOB) samples by model uncertainty and, within each group, selects the structure of an OOB residual corrector by cross-validation over a small discrete catalog. Only this structure is selected; all numerical parameters and the final correction strength are fitted directly from data. Our main contributions are:

- We introduce a method that evolves an executable, dataset-specific split-scoring program and uses it throughout tree construction in place of a fixed criterion such as Gini impurity or variance reduction.
- We propose an uncertainty-aware OOB residual correction mechanism for the trained forest. EvoGraft uses out-of-bag predictions to identify systematic prediction errors in uncertainty buckets and selects the structure of each bucket's corrector by cross-validation on its out-of-bag residuals, while all numerical parameters are fitted from data.
- We evaluate EvoGraft against tree-based and LLM-based methods on 14 CARTE datasets, complemented by full-data and few-shot split-criterion comparisons on OpenML, cross-domain transfer, feature engineering, and component ablations. EvoGraft achieves the lowest mean rank in both CARTE tasks under the tested configurations.

2 Related Work

2.1 LLM for Tabular Data

LLMs for feature engineering. A growing line of work uses LLMs to improve the representation of tabular data before prediction. CAAFE [18] generates context-aware features from dataset descriptions and evaluates them with downstream models. FeatLLM [16] generates feature rules from a few labeled examples. These approaches improve the feature space while leaving prediction to a downstream model. EvoGraft addresses the unchanged downstream split objective by evolving a dataset-specific scoring program.

LLMs as zero-shot and few-shot learners. Another direction uses LLMs directly as learners for tabular data. LIFT [11] fine-tunes language models on inputs and outputs serialized as text. TabLLM [17] serializes rows into natural language for few-shot classification. TABLET [34] studies how task instructions help zero- and few-shot prediction. P2T [29] builds prompt demonstrations from heterogeneous source tables, and InsightTab [39] distills insights from training examples for few-shot classification. These methods rely on pretrained knowledge, task descriptions, and a limited number of examples. EvoGraft avoids inference-time LLM calls by deploying the evolved forest and data-fitted correctors.

LLMs for tree and forest construction. More recent work combines LLMs with decision trees and tree ensembles. LLM Trees [23] study zero-shot decision-tree induction and embedding, directly eliciting tree structures from LLMs. LLEGO [26] uses LLMs together with evolutionary search to guide decision-tree induction, allowing semantic information from the task to influence how trees are constructed. MAPLE [21] uses multiple LLM agents to construct tree ensembles, with different agents contributing to the tree-building process. FORESTLLM [36] combines LLM knowledge with random forests for few-shot tabular learning and uses LLM-derived information during tree splitting and leaf prediction. DeLTa [37] takes a different approach by starting from an existing tabular predictor and using an LLM-generated rule to correct its residual prediction errors. EvoGraft addresses both unchanged split objectives and residual errors by combining split-score evolution with OOB correction.

2.2 Automated Heuristic Design

Automated heuristic design (AHD) aims to automatically discover executable rules or algorithms for optimization problems. Recent work uses LLMs as program generators inside a search loop, where candidate heuristics are generated, evaluated, and improved using performance feedback. FunSearch [33] combines an LLM with evolutionary program search to repeatedly generate and select high-performing programs. Evolution of Heuristics (EoH) [25] further develops this idea by using LLMs with evolutionary operators to generate and refine heuristic algorithms. Several methods improve how the search process uses feedback. ReEvo [38] introduces reflective evolution, where the LLM compares better and worse heuristics and uses the resulting feedback to guide later generations. HSEvo [10] focuses on search diversity by combining LLM-based generation with harmony search and genetic algorithm mechanisms. MCTS-AHD [41] instead organizes heuristic generation with Monte Carlo Tree Search, providing a more structured balance between exploring new candidates and improving promising ones. HiFo-Prompt [7] further improves heuristic generation by using hindsight from previous solutions and foresight about the current search state to adjust the prompting strategy.

3 Methodology

Figure 1 presents the two stages of EvoGraft. The first stage evolves a dataset-specific split-scoring program using predictive fitness and symbolic experience extracted from the induced forests. The frozen best program is then used to construct an ensemble whose OOB evidence supports residual adaptation with cross-validated structure selection, data-driven parameter fitting, and held-out calibration in the second stage.

3.1 Problem Formulation

Problem Setup. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ denote a tabular dataset with N samples and p input features, where $\mathbf{x}_i \in \mathbb{R}^p$. For classification, $y_i \in \{1, \dots, C\}$ with C classes, while for regression, $y_i \in \mathbb{R}$. We seek to learn an ensemble of K decision trees, $\mathcal{F} = \{T_k\}_{k=1}^{K}$, that maps an input $\mathbf{x}$ to a prediction and generalizes to unseen data. Each decision tree [5] partitions the feature space into regions and assigns a prediction to each terminal leaf. For classification, each tree produces a class-probability vector $T_k(\mathbf{x}) \in [0,1]^C$, while for regression it produces a scalar prediction $T_k(\mathbf{x}) \in \mathbb{R}$. The ensemble prediction is obtained by aggregating predictions across its trees.

Bagging and OOB Evaluation. Each tree T_k is trained on a bootstrap sample $\mathcal{D}^{(k)}$ drawn with replacement from $\mathcal{D}$, while samples not selected in $\mathcal{D}^{(k)}$ form its out-of-bag (OOB) set O_k [4]. For each training sample i, let $\mathcal{K}_i = \{k \in \{1, \dots, K\} : i \in O_k\}$ denote the set of trees for which sample i is OOB. Its OOB prediction averages only these trees (a class-probability vector for classification, a scalar for regression). Because sample i is not used to train any tree in $\mathcal{K}_i$, OOB predictions estimate predictive behavior on unseen samples without an additional validation split.

3.2 Experience-Guided Split-Score Evolution

Consider an internal node v containing the training samples that reach that node. For each candidate split $s \in \mathcal{S}_v$, let $\mathbf{y}_{v,s}^{L}$ and $\mathbf{y}_{v,s}^{R}$ denote the target values assigned to the left and right child, respectively. Instead of ranking candidates using a fixed impurity criterion, EvoGraft represents the split criterion itself as an executable program g and defines

$$S_g(v,s) = g\Big(\mathbf{y}_{v,s}^{L}, \mathbf{y}_{v,s}^{R}, \mathbf{c}_{v,s}\Big), \qquad s_v^{*} = \arg\max_{s \in \mathcal{S}_v} S_g(v,s). \tag{1}$$

The same program g is applied throughout tree construction, while its behavior may vary across nodes and candidate splits through the context $\mathbf{c}_{v,s}$, which includes node depth and sample fraction, left and right child fractions and split balance, previous use of the candidate feature, features appearing on the root-to-parent path, and the number of input features or classes. The context contains no validation or test labels, predictions, or fitness values, so g ranks each candidate using only information available during tree construction.

A candidate split-scoring program is evaluated through the predictive performance of the tree structures it induces rather than through the quality of individual splits in isolation. Let $\mathcal{M}$ denote a set of development runs. For each run $\ell \in \mathcal{M}$, a model $\mathcal{F}_g^{\ell}$ is trained using g to rank candidate splits throughout tree construction, and its predictive quality is measured on the corresponding development data $\mathcal{D}_{\text{dev}}^{\ell}$. We define

$$\mathcal{J}(g) = \frac{1}{|\mathcal{M}|} \sum_{\ell \in \mathcal{M}} Q\Big(\mathcal{F}_g^{\ell}, \mathcal{D}_{\text{dev}}^{\ell}\Big), \qquad \widetilde{\mathcal{J}}(g) = \rho\, \mathcal{J}(g), \tag{2}$$

where Q is the task-specific evaluation metric. We set $\rho = 1$ for metrics to maximize, such as accuracy, AUROC, and R^2, and $\rho = -1$ for

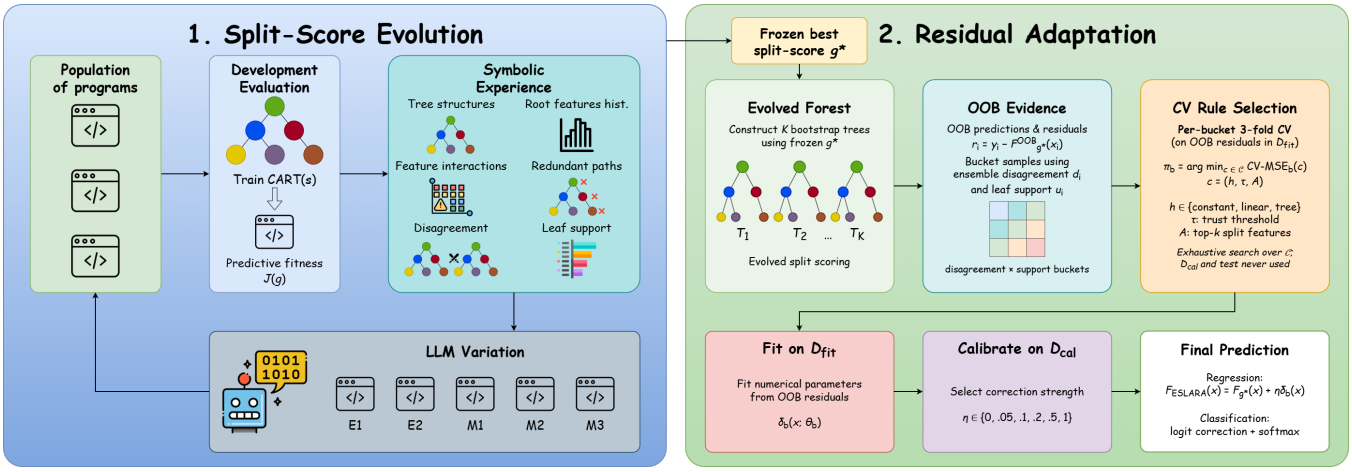


Figure 1: Overview of EvoGraft. Split-score evolution uses predictive fitness and symbolic forest experience to evolve a dataset-specific scoring program $g^\star$. The resulting forest provides OOB residual and uncertainty evidence from which cross-validation selects correction structures; their numerical parameters are fitted from data and their strength is calibrated before final prediction.

metrics to minimize, such as log-loss, MAE, and RMSE. Therefore, larger values of $\widetilde{\mathcal{J}}(g)$ are always preferred during evolution.

Initialize and evaluate. Let P and E denote the population and elite-pool sizes. The initial population combines reference splitting criteria with randomly generated programs. Each individual pairs a natural-language rationale with an executable `score_split` function. Valid programs are evaluated by retraining the development models, ranked by $\widetilde{\mathcal{J}}$, and truncated to the top P unique individuals.

Extract symbolic experience. For each of the top E individuals, EvoGraft retrains its development model and distills the induced forest into a textual experience report. Each report contains (i) meta-information about the task, including the feature inventory with numeric and categorical counts, the number of trees, and the training and validation sizes; (ii) forest-level diagnostics: the histogram of root-split features, which exposes dominant and under-explored features, the most frequent feature-interaction pairs co-occurring along root-to-leaf paths, clusters of redundant paths grouped by the Jaccard similarity of their validation coverage, and the validation samples with the highest cross-tree prediction disagreement, listed together with the leaf rules that produced each prediction; and (iii) the best and worst representative trees, serialized with their per-split scores and node diagnostics (depth, leaf count, feature reuse along branches, and leaf support skew), accompanied by derived recommendations on which behaviors to prefer or avoid. All statistics are computed exclusively from development training and validation data.

Select parents and generate offspring. Parents are sampled with replacement from the ranked population using

$$p_r = \frac{(P + r + 1)^{-1}}{\sum_{j=0}^{P-1}(P + j + 1)^{-1}}, \tag{3}$$

where $r \in \{0, 1, \ldots, P-1\}$ is the rank of an individual in the population sorted in descending order of $\widetilde{\mathcal{J}}$. The recombination operators E1 and E2 each sample several parents, whereas the mutation operators M1–M3 act on a single parent. E1 explores globally by requesting a heuristic with a substantially different mathematical form from its parents. E2 recombines parents by preserving their well-supported successful splits, altering the purity-versus-balance trade-off at weak nodes, and deriving a new form from observed failure patterns rather than copying any parent. M1 repairs failures by studying the reported validation errors, weak leaves, and symbolic tree paths to address one supported failure pattern while preserving successful behavior. M2 tunes constants by re-weighting the candidate-dependent purity, balance, and small-child penalty terms instead of rescaling the whole score. M3 simplifies by identifying which of purity gain, partition balance, or split information is over-emphasized and rebuilding the heuristic around a candidate-dependent trade-off. The symbolic experience extracted in the previous step accompanies every operator: all five prompts receive the parent programs, their fitness, and the elite experience text.

Validate and select survivors. Unsafe, invalid, duplicate, or non-finite offspring are rejected. Valid offspring are evaluated by retraining the development models. Parents and offspring are merged, ranked, deduplicated, and truncated to the best P programs, yielding elitist $(\mu + \lambda)$ selection.

The loop terminates at the generation or evaluation budget. The best program

$$g^\star = \arg\max_{g \in \mathcal{G}} \widetilde{\mathcal{J}}(g) \tag{4}$$

over the set $\mathcal{G}$ of evaluated programs is frozen and used for all subsequent forest construction.

3.3 OOB Residual Adaptation

The frozen score $g^\star$ determines *where* trees split; this second stage adapts the forest *after* fitting, using out-of-bag (OOB) residuals.

Forest construction. Each of the K trees is trained on an independent bootstrap replica of the training data, and every split of every tree is scored by the frozen program $g^\star$ instead of a fixed CART criterion. The evolved forest and its OOB counterpart are

$$\widehat{F}_{g^\star}(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} T_k(\mathbf{x}), \qquad \widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i) = \frac{1}{|\mathcal{K}_i|}\sum_{k\in\mathcal{K}_i} T_k(\mathbf{x}_i), \quad (5)$$

with $\mathcal{K}_i$ as defined above ($\widehat{F}_{g^\star}$ is used if $\mathcal{K}_i = \emptyset$).

Uncertainty-aware bucketing. Let $\mathcal{D}_{\mathrm{cor}}$ denote the OOB correction data. We define the OOB correction target as

$$\mathbf{r}_i = \begin{cases} y_i - \widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i), & \text{regression,} \\ \mathbf{e}_{y_i} - \widehat{\mathbf{p}}^{\mathrm{OOB}}_{g^\star,i}, & \text{classification,} \end{cases} \quad (6)$$

where $\widehat{\mathbf{p}}^{\mathrm{OOB}}_{g^\star,i} = \widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i)$ is the OOB class-probability vector and $\mathbf{e}_{y_i}$ is the one-hot encoding of class y_i; for classification, $\mathbf{r}_i$ is thus the negative gradient of the cross-entropy with respect to the logits, while for regression it is scalar-valued. Two uncertainty indicators are attached to every sample: the ensemble disagreement $d_i = \frac{1}{KC}\sum_{k=1}^{K}\left\|T_k(\mathbf{x}_i) - \widehat{F}_{g^\star}(\mathbf{x}_i)\right\|^2$, the mean squared deviation of individual-tree predictions from the forest prediction ($C = 1$ for regression), and the leaf support $u_i = \min_{1\le k\le K} s_k(L_k(\mathbf{x}_i))$, where $s_k(L_k(\mathbf{x}_i))$ is the training-sample count of the leaf of T_k reached by $\mathbf{x}_i$; it flags samples routed through thin, memorized regions of the trees. Each sample is assigned to a bucket $b_i = \big(\phi_d(d_i), \phi_u(u_i)\big)$ of an $m \times m$ grid, where ϕ_d and ϕ_u map d_i and u_i to quantile bins; the bin edges are estimated from the fitting subset defined below only, so calibration labels never influence the bucketing.

Correction structure. Each bucket b receives a policy $\pi_b = (h_b, \tau_b, \mathcal{A}_b)$, where $h_b \in \{\textsc{constant}, \textsc{linear}, \textsc{tree}\}$ is the corrector family, $\tau_b \in \mathcal{T} = \{0.5, 0.7, 0.9, 1\}$ is a within-bucket disagreement-rank threshold, and $\mathcal{A}_b$ is a feature subset. Features are ranked per bucket by the fraction of trees that split on them along the decision paths of the bucket's samples, and $\mathcal{A}_b$ is restricted to the top-a_b prefix of this ranking, with $1 \le a_b \le p_{\max}$ for the linear and tree families and $a_b = 0$ for the constant family. Every bucket therefore chooses from the same catalog Π of $|\mathcal{T}|(1 + 2p_{\max})$ policies. Only this discrete structure is selected; every numerical parameter is fitted from OOB residuals.

Cross-validated structure selection. The policy of each bucket is chosen by cross-validation on its own OOB residuals. For every candidate $\pi \in \Pi$, the rows of $\mathcal{I}_b$ (defined below) are split into three folds; a corrector with structure π is fitted on two folds with the trust weights and shrinkage of the next paragraph and predicts the residuals of the held-out fold. The bucket keeps the policy with the lowest cross-validated residual MSE,

$$\pi_b = \arg\min_{\pi\in\Pi} \frac{1}{n_b C}\sum_{f=1}^{3}\sum_{i\in\mathcal{I}_b^{(f)}} \left\|\mathbf{r}_i - \widehat{\boldsymbol{\delta}}_{\pi}^{(-f)}(\mathbf{x}_i)\right\|^2, \quad (7)$$

where $\mathcal{I}_b^{(f)}$ is fold f and $\widehat{\boldsymbol{\delta}}_{\pi}^{(-f)}$ is fitted on the other two, with ties broken toward simpler families, fewer features, and larger τ. Buckets with fewer than four rows keep a fixed default policy.

Data-fitted residual correction. We split $\mathcal{D}_{\mathrm{cor}}$ into a fitting subset $\mathcal{D}_{\mathrm{fit}}$ and a disjoint calibration subset $\mathcal{D}_{\mathrm{cal}}$ (25%, stratified for classification). Both are training rows of the forest: $\mathcal{D}_{\mathrm{cal}}$ is held out from bucketing, structure selection, and corrector fitting, but not from the bootstrap samples, so it is evaluated only through OOB predictions. Let $\mathcal{I}_b = \{i \in \mathcal{D}_{\mathrm{fit}} : b_i = b\}$ and $n_b = |\mathcal{I}_b|$. The within-bucket disagreement rank and trust weight are

$$q_i^{(b)} = \frac{1}{n_b}\sum_{j\in\mathcal{I}_b} \mathbf{1}[d_j \le d_i], \qquad \alpha_i = \begin{cases} 1, & q_i^{(b)} \le \tau_b, \\ 0.3, & q_i^{(b)} > \tau_b. \end{cases} \quad (8)$$

so that samples whose disagreement exceeds the bucket's trusted rank contribute only with weight 0.3. Let $\mathbf{W}_b = \mathrm{diag}(\alpha_i)$, $\mathbf{v}_i = [1, \mathbf{x}_{i,\mathcal{A}_b}^\top]^\top$, and let $\mathbf{V}_b$ and $\mathbf{R}_b$ stack $\mathbf{v}_i^\top$ and $\mathbf{r}_i^\top$, respectively. The bucket-specific corrector is

$$\widehat{\boldsymbol{\delta}}_b(\mathbf{x}) = \begin{cases} \dfrac{n_b \bar{\mathbf{r}}_b^{\alpha} + \lambda_{\mathrm{sh}}\bar{\mathbf{r}}}{n_b + \lambda_{\mathrm{sh}}}, & h_b = \textsc{constant}, \\ \mathbf{v}(\mathbf{x})^\top \widehat{\mathbf{B}}_b, & h_b = \textsc{linear}, \\ T_b^{\mathrm{res}}(\mathbf{x}_{\mathcal{A}_b}), & h_b = \textsc{tree}, \end{cases} \quad (9)$$

where

$$\begin{aligned} \bar{\mathbf{r}}_b^{\alpha} &= \frac{\sum_{i\in\mathcal{I}_b}\alpha_i \mathbf{r}_i}{\sum_{i\in\mathcal{I}_b}\alpha_i}, \\ \bar{\mathbf{r}} &= \frac{1}{|\mathcal{D}_{\mathrm{fit}}|}\sum_{i\in\mathcal{D}_{\mathrm{fit}}} \mathbf{r}_i, \\ \widehat{\mathbf{B}}_b &= (\mathbf{V}_b^\top \mathbf{W}_b \mathbf{V}_b)^{\dagger} \mathbf{V}_b^\top \mathbf{W}_b \mathbf{R}_b. \end{aligned} \quad (10)$$

The constant corrector shrinks the trust-weighted bucket mean toward the global mean residual with strength λ_{sh}, which stabilizes small buckets; the linear corrector is a trust-weighted least-squares fit with an intercept, using the pseudo-inverse for rank-deficient designs; the tree corrector T_b^{res} is a shallow regression tree (depth 3, minimum leaf size 5) trained on $\mathcal{A}_b$ with the same weights. The strength $\eta \in \{0, 0.05, 0.1, 0.2, 0.5, 1\}$ is then chosen by the MSE (regression) or log-loss (classification) on $\mathcal{D}_{\mathrm{cal}}$ of the OOB-based corrected prediction $\widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i) + \eta\widehat{\boldsymbol{\delta}}_{b_i}(\mathbf{x}_i)$, applied in logit space for classification as below; $\eta = 0$ lets calibration switch the correction off. With $b(\mathbf{x})$ the bucket of $\mathbf{x}$ and $\widehat{\boldsymbol{\delta}}_b = \mathbf{0}$ for buckets without fitting rows, the regression predictor is $\widehat{F}_{\mathrm{EvoGraft}}(\mathbf{x}) = \widehat{F}_{g^\star}(\mathbf{x}) + \eta\widehat{\boldsymbol{\delta}}_{b(\mathbf{x})}(\mathbf{x})$. For classification, the correction is instead applied in logit space: $\widehat{\mathbf{p}}_{\mathrm{EvoGraft}}(\mathbf{x}) = \mathrm{softmax}\Big(\mathbf{z}_{g^\star}(\mathbf{x}) + \eta\widehat{\boldsymbol{\delta}}_{b(\mathbf{x})}(\mathbf{x})\Big)$, where $\mathbf{z}_{g^\star} = \log\widehat{\mathbf{p}}_{g^\star} - \frac{1}{C}\sum_c \log\widehat{p}_{g^\star,c}$ is the centered-logit representation of the evolved forest probabilities; correcting in logit space rather than probability space keeps the corrected vector on the simplex after the final softmax, and $\widehat{\boldsymbol{\delta}}_b$ then has one component per class.

4 Experimental Results

4.1 Experimental Setting

Datasets and metrics. We evaluate EvoGraft on 14 datasets drawn from the CARTE benchmark [22], comprising seven classification tasks and seven regression tasks (Tables 1 and 2), which mix numerical and high-cardinality categorical attributes. We report macro-F1 for classification and root mean squared error (RMSE) for regression, computed on the transformed target scale when preprocessing transforms the target.

Data split. For each dataset, we first reserve 20% of the data as an *evolution pool*, used exclusively to search for the dataset-specific split-scoring program. This pool is divided into 75% development-training data, on which the candidate models are fitted, and 25% development-validation data, on which their evolutionary fitness is measured. The remaining 80% forms the *benchmark pool* shared by EvoGraft and all baselines. For each dataset, we use a fixed 75/25 train/test split of this pool, shared by all methods. We repeat training ten times while keeping this split fixed, varying the randomness used during training. The held-out benchmark test data are never used during split-score evolution, model fitting, residual-corrector selection, or calibration.

Compared baselines. We compare EvoGraft with baselines from three groups: (i) LLM-based methods, including DeLTa [37], MAPLE [21], and, for classification only, LLM-Tree [23] and FeatLLM [16]; (ii) tree-based methods, including Extra Trees (ET) [13], Random Forest (RF) [4], XGBoost (XGB) [8], and LightGBM (LGBM) [20]; and (iii) the tabular foundation model TabPFN-v2 [19].

4.2 Main Results

Tables 1 and 2 report macro-F1 and RMSE on the fourteen CARTE datasets. For each dataset we rank all methods with results by their mean score and report the mean rank across datasets.

Classification. EvoGraft has the lowest mean rank and the highest mean macro-F1 on Chocolate and Coffee. Its advantage is clearest on Coffee, where it exceeds the strongest baseline, LightGBM, by 5.4 percentage points. It remains below MAPLE on Michelin and Yelp, DeLTa on US Accidents, and both boosted ensembles on Whisky. On Spotify, its mean score is close to those of DeLTa and TabPFN-v2. These results support competitive classification performance across the tested tables, with specific weaknesses that the aggregate rank does not capture. FeatLLM and LLM-Tree show greater variation across training runs on most tables. These deviations reflect variability in training on a fixed data partition; the split-scoring programs remain frozen.

Regression. EvoGraft has the lowest mean rank and lower mean RMSE than DeLTa and the four tree ensembles on all seven regression datasets under the tested configurations. It leads on Cardekho and Museums. TabPFN-v2 has lower mean RMSE on four datasets, while MAPLE performs better on Anime Planet, Used Cars, and Wine Prices. EvoGraft therefore improves on the evaluated conventional tree baselines without replacing the strongest alternatives across all regression tasks.

4.3 Evolution Dynamics

Figure 2 traces the split-program search on the CARTE datasets. For every dataset and generation, it reports the fitness of the best program found so far relative to the best hand-designed criterion in the initial population, which contains CART, ID3, and C4.5-style criteria and balanced or hybrid variants of them. Fitness is measured on the development-validation split of the evolution pool. Relative to the best hand-designed initial criterion, the selected programs increase macro-F1 by 6.4% on average across classification datasets and reduce RMSE by 4.4% across regression datasets on this split. Over half of the average gain is reached within the first five generations, while later generations add smaller gains. The gain varies from under 1% on Used Cars to about 12% on Yelp, indicating that the search is more useful on some development splits than others. Because selection repeatedly uses these data and retains the best program found so far, the rising curves show progress on the search objective rather than independent evidence of better generalization.

4.4 Effect of the Evolved Split Criterion

To isolate the split-scoring program from the forest and the residual stage, we grow a single decision tree with the same tree builder and replace only its split criterion. We use seven numeric multi-class datasets with 6 to 26 classes, all obtained from the OpenML repository [35]: JapaneseVowels, Letter, Optdigits, Pendigits, Satimage, and Segment, which originate from the UCI repository [12], and Texture. We repeat training ten times, using either the full training set or $k \in \{128, 64, 32, 16\}$ labeled examples per class sampled from it. We compare the evolved program with four hand-designed criteria [27]: Gini impurity [5], entropy (information gain) [30], gain ratio [31], and class-weighted entropy, which weights each class by its inverse frequency at the parent node, analogous to the altered class priors of [5], and report each criterion's mean accuracy (Figure 3). The evolved program achieves higher mean accuracy than all evaluated human-designed criteria across the tested budgets, including the few-shot settings.

We hypothesize that this few-shot performance stems from the structural information available to the LLM through symbolic experience. Reports of representative trees, root-to-leaf feature interactions, weak leaves, and validation errors allow the LLM to relate split decisions to the behavior of the induced trees. When labeled examples are scarce, local impurity estimates can be noisy; this feedback may encourage scoring programs that balance purity with partition support and tree context, providing a useful inductive bias for learning from few examples.

We repeat the comparison with random forests of 100 trees of depth 6 on the same training subsets, with $k \in \{16, 32, 64, 128\}$ labeled examples per class and on the full training split (Figure 4); both forests share the tree builder and differ only in the split criterion. Replacing Gini impurity with the evolved program raises mean accuracy at every budget, by 0.8–1.6 percentage points, with gains on all seven datasets at 64 and 128 examples per class. The gains are smaller than for a single tree, as bagging already reduces the variance of individual splits.

4.5 Transfer of Split Programs Across Domains

We next ask whether a split program evolved on one dataset remains useful on datasets from a different domain. We take the seven programs evolved on the CARTE classification datasets and the seven evolved on the CARTE regression datasets and deploy each of them, without further search, on seven OpenML classification datasets (Vehicle, Steel Plates, Mfeat-Morphological, CMC, Eucalyptus, Vowel, and Credit-g) and seven OpenML regression datasets (Airfoil, Concrete, QSAR Fish, Space GA, White Wine, Geo Music, and House 16H, in the versions released with the DeLTa benchmark [37]). Unlike the CARTE tables, these datasets are mostly

Table 1: Macro-F1 (mean ± SD over ten training runs on a fixed train/test split; FeatLLM completed eight runs on Coffee and nine on Michelin/Spotify). Bold/underlined: best/second-best; lower rank is better.

Dataset	EvoGraft	LLM-Tree	DeLTa	FeatLLM	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Chocolate	**0.597±0.011**	0.403±0.094	0.574±0.024	0.448±0.078	0.589±0.018	0.371±0.010	0.374±0.005	0.533±0.007	0.532±0.005	0.561±0.032
Coffee	**0.580±0.037**	0.448±0.075	0.502±0.065	0.487±0.099	0.383±0.008	0.376±0.003	0.379±0.007	0.525±0.012	0.526±0.004	0.378±0.005
Michelin	0.698±0.023	0.473±0.088	0.586±0.155	0.568±0.062	**0.715±0.009**	0.662±0.026	0.682±0.016	0.673±0.004	0.674±0.005	0.704±0.017
Spotify	0.776±0.001	0.479±0.119	**0.777±0.004**	0.726±0.006	0.770±0.003	0.719±0.032	0.725±0.016	0.768±0.001	0.769±0.001	0.776±0.007
US Accidents	0.698±0.006	0.429±0.075	**0.710±0.012**	0.609±0.050	0.706±0.004	0.620±0.024	0.620±0.025	0.698±0.001	0.699±0.002	0.625±0.027
Whisky	0.755±0.010	0.518±0.107	0.712±0.028	0.614±0.167	0.725±0.020	0.723±0.023	0.707±0.020	0.774±0.008	**0.779±0.004**	0.758±0.023
Yelp	0.650±0.001	0.443±0.072	0.642±0.050	0.611±0.015	**0.662±0.006**	0.349±0.001	0.349±0.001	0.643±0.002	0.644±0.002	0.638±0.011
Mean rank ↓	**2.43**	8.86	4.14	7.57	3.14	8.43	7.71	4.43	3.57	4.71

Table 2: RMSE (mean ± SD over ten training runs on a fixed train/test split). Bold/underlined: best/second-best; lower RMSE and rank are better.

Dataset	EvoGraft	DeLTa	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Anime Planet	0.669±0.011	0.731±0.028	0.664±0.006	0.839±0.005	0.850±0.005	0.691±0.001	0.692±0.001	**0.656±0.019**
Beer Ratings	0.317±0.003	0.475±0.032	0.332±0.004	0.391±0.003	0.401±0.002	0.347±0.000	0.345±0.001	**0.302±0.005**
Cardekho	**0.073±0.000**	0.075±0.001	0.078±0.000	0.145±0.004	0.149±0.003	0.104±0.000	0.104±0.000	0.095±0.010
Employee Sal.	0.099±0.003	0.120±0.007	0.101±0.002	0.152±0.003	0.153±0.003	0.122±0.000	0.122±0.000	**0.084±0.005**
Museums	**0.479±0.007**	0.530±0.050	0.504±0.003	0.614±0.004	0.615±0.004	0.504±0.001	0.504±0.001	0.496±0.027
Used Cars	0.112±0.003	0.121±0.006	**0.106±0.001**	0.158±0.002	0.166±0.002	0.126±0.000	0.126±0.000	0.130±0.005
Wine Prices	0.728±0.010	0.780±0.028	0.724±0.009	1.032±0.007	1.043±0.008	0.842±0.000	0.850±0.000	**0.667±0.016**
Mean rank ↓	**2.00**	4.71	2.71	6.86	7.86	4.86	4.71	2.29

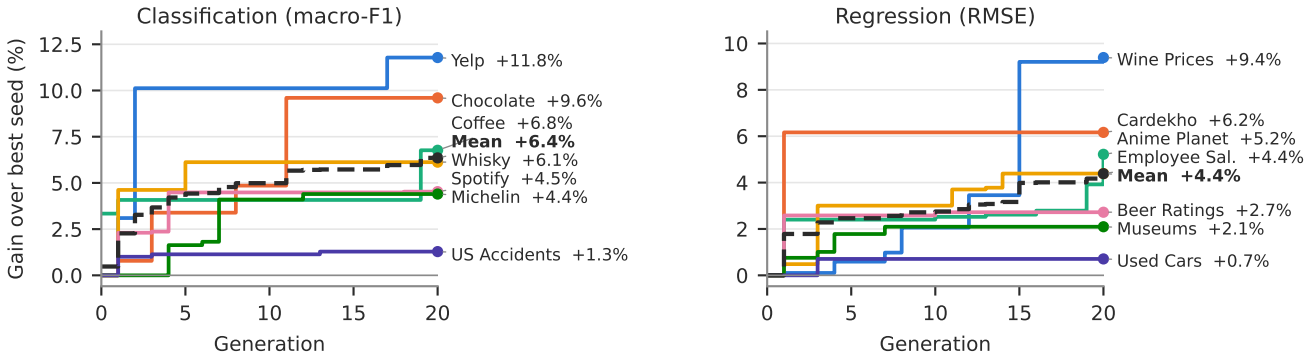


Figure 2: Split-program evolution on the seven classification (left) and seven regression (right) CARTE datasets: improvement of the best program found so far over the best hand-designed criterion in the initial population, on the development-validation split of the evolution pool. Each coloured line is one dataset, labelled at its right end with its final gain; the dashed line is the mean over datasets.

numeric, and they differ in size and in the number of classes. Figure 5 reports the differences averaged over ten training runs on each target dataset.

Transfer is uneven across datasets. In classification, all transferred programs improve performance on Steel Plates but reduce it on CMC, while their effects vary on Mfeat-Morphological and Eucalyptus. In regression, transfer provides no aggregate improvement: the programs reduce error on average on Airfoil but increase it on Concrete, Space GA, and House 16H. The outcome also depends on the source–target pair: on Concrete, the program evolved on Museums reduces error, while the Wine Prices program increases it. These results highlight the importance of evolving split-scoring programs separately for each dataset, so that split decisions reflect its statistical and structural characteristics.

4.6 Split Programs with Engineered Features

The split programs are evolved on the original attributes of each dataset. We therefore ask how EvoGraft performs when its forest is grown on a richer feature set produced by feature engineering. We consider two LLM-based generators, LLM-FE [1], which evolves

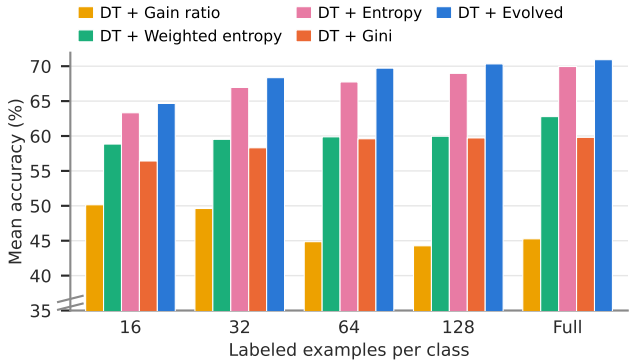


Figure 3: Single-tree accuracy on seven OpenML datasets, averaged over ten training runs, for k labels per class and full training data, with a decision tree (DT) grown with each split criterion.

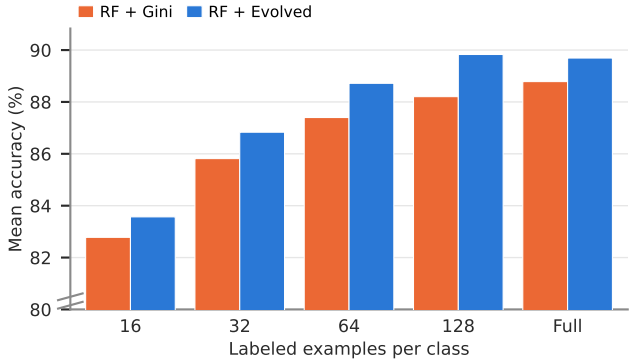


Figure 4: Random-forest (RF) accuracy on seven OpenML datasets, averaged over ten training runs, comparing Gini and evolved scores with k labels per class or full training data.

a feature-transformation program, and OCTree [28], which proposes rule-based features, and OpenFE [40], an automated feature generator that does not use an LLM. Each generator is run once per dataset on the fixed training split, and its features are frozen and reused in all ten training runs. EvoGraft is then fitted on the resulting features with the same split program and residual stage as in Tables 1 and 2, without further evolution (Table 3). LLM-FE and OpenFE generally improve EvoGraft, whereas OCTree provides no consistent benefit, suggesting that gains depend on feature quality. The evolved split programs can exploit the richer inputs without further evolution. These results suggest that feature engineering and split-program evolution are complementary: one enriches the inputs, while the other guides their partitioning, allowing EvoGraft to benefit from both.

Table 3: Feature engineering with EvoGraft on CARTE (mean ± SD over ten training runs on a fixed train/test split). Bold/underlined: best/second-best.

Dataset	EvoGraft	+ LLM-FE	+ OCTree	+ OpenFE
Classification: macro-F1 (higher is better)				
Chocolate	0.597±0.011	**0.680±0.008**	0.577±0.011	0.654±0.014
Coffee	0.580±0.037	**0.752±0.009**	0.510±0.047	0.624±0.022
Michelin	0.698±0.023	**0.780±0.008**	0.714±0.018	0.734±0.007
Spotify	0.776±0.001	0.773±0.002	0.777±0.002	**0.851±0.002**
US Accidents	0.698±0.006	**0.715±0.004**	0.696±0.006	0.712±0.004
Whisky	0.755±0.010	**0.773±0.009**	0.757±0.009	0.765±0.012
Yelp	0.650±0.001	**0.710±0.001**	0.646±0.002	0.697±0.002
Regression: RMSE (lower is better)				
Anime Planet	0.669±0.011	**0.613±0.006**	0.665±0.013	0.629±0.002
Beer Ratings	**0.317±0.003**	0.329±0.002	0.317±0.004	0.318±0.003
Cardekho	0.073±0.000	**0.067±0.002**	0.072±0.000	0.071±0.000
Employee Sal.	0.099±0.003	0.096±0.002	0.102±0.002	**0.095±0.002**
Museums	0.479±0.007	**0.412±0.002**	0.492±0.021	0.438±0.009
Used Cars	0.112±0.003	**0.084±0.001**	0.113±0.004	0.105±0.002
Wine Prices	0.728±0.010	**0.693±0.008**	0.721±0.005	0.698±0.007

4.7 Ablation Study

Table 4 changes one design choice of EvoGraft at a time on the seven OpenML classification datasets of Section 4.4 and the seven CARTE regression datasets. Each variant is evaluated over ten training runs, with matching training and test data across variants.

Residual corrector. We remove the residual stage and evaluate the evolved forest on its own. Performance deteriorates in both classification and regression, indicating that systematic prediction errors remain after split-score evolution. OOB residual correction therefore complements the evolved tree construction by addressing errors in the forest's aggregated predictions.

Evolved split score. We keep the corrector and replace the evolved program with Gini impurity for classification and variance reduction for regression. In classification, the evolved score yields higher mean accuracy, and in regression it yields lower mean RMSE, even when residual correction is retained. This shows that adapting the split objective contributes beyond post-hoc correction, consistent with Section 4.4, and supports adapting both tree construction and forest-level prediction.

Structure selection. We replace the cross-validated choice of each bucket's corrector structure (Eq. 7) by a random choice from Π or by a heuristic that uses all top-$p_{\max}$ features, $\tau_b = 1$, and a constant or linear corrector depending only on the bucket size. Both alternatives yield lower mean accuracy and higher mean RMSE than cross-validation, with a clearer benefit from cross-validation in regression. Its frequent selection of tree correctors, which the heuristic excludes, suggests that flexibility in the correction family is useful. These results support selecting correction structures from OOB evidence to match the residual patterns of each bucket.

Bucket grid size. We vary the grid size m, which sets how many uncertainty buckets receive their own corrector; $m = 1$ fits a single corrector for the whole forest and EvoGraft uses $m = 3$. Multiple buckets outperform a single global corrector in both tasks. These

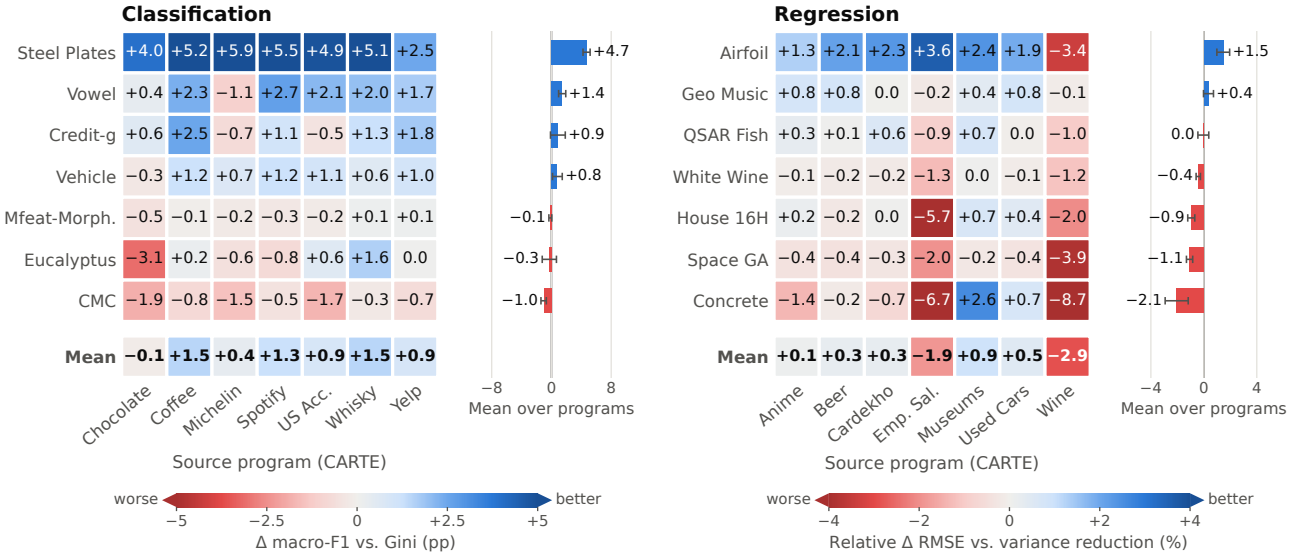


Figure 5: CARTE-to-OpenML transfer gains over Gini/variance reduction, averaged over ten training runs. Columns/rows: sources/targets; side bars: target means ± SE; bottom rows: source means.

results support adapting correction to different uncertainty regions, with the appropriate granularity depending on the task.

Table 4: Ablation: mean accuracy/RMSE on OpenML/CARTE over ten training runs. Gini: fixed Gini/variance reduction. Bold/underlined: best/second-best.

Ablation	Variant	Accuracy ↑	RMSE ↓
Corrector	Removed	0.832	0.374
Split score	Gini	0.836	0.356
Structure	Random	0.840	0.367
	Heuristic	0.847	0.364
Grid size	$m = 1$	0.837	0.357
	$m = 2$	0.845	0.356
EvoGraft (evolved, CV, $m = 3$)		**0.850**	**0.354**

5 Conclusion

We presented EvoGraft, a framework that adapts tree construction through dataset-specific split-score evolution and forest predictions through uncertainty-aware OOB residual correction. Symbolic experience connects LLM-generated scoring programs to the structures and errors of the induced trees, while OOB evidence guides correction after forest fitting. On fourteen tabular benchmarks, EvoGraft obtains the lowest mean rank among the evaluated methods in both classification and regression under the tested configurations. Its advantages remain dataset-dependent, and uneven transfer highlights the importance of evolving split scores for individual datasets.

Several limitations remain. First, split-score evolution requires repeated model fitting and LLM calls, adding a search cost whose usefulness depends on the available budget and the quality of generated programs. Second, repeated selection on development-validation data may favor programs tailored to those data, and a program evolved on one dataset may not transfer reliably to another. Finally, correctors are restricted to a fixed catalog of constant, linear, and shallow tree models, discrete trust thresholds, and ranked feature subsets.

Future work could improve search efficiency, evaluate programs across more varied development splits, and adapt bucket granularity to the available OOB evidence. Expanding or evolving the set of correction models is another direction, while preserving the separation between structure selection, parameter fitting, and calibration.

References

[1] Nikhil Abhyankar, Parshin Shojaee, and Chandan K. Reddy. 2026. LLM-FE: Automated Feature Engineering for Tabular Data with LLMs as Evolutionary Optimizers. arXiv:2503.14434 [cs.LG] https://arxiv.org/abs/2503.14434

[2] Rodrigo C. Barros, Márcio P. Basgalupp, André C. P. L. F. de Carvalho, and Alex A. Freitas. 2012. A Hyper-Heuristic Evolutionary Algorithm for Automatically Designing Decision-Tree Algorithms. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*. ACM, 1237–1244. doi:10.1145/2330163.2330335

[3] Márcio Porto Basgalupp, Rodrigo Coelho Barros, and Tiago Barabasz. 2014. A Grammatical Evolution Based Hyper-Heuristic for the Automatic Design of Split Criteria. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*. ACM, 1311–1318. doi:10.1145/2576768.2598327

[4] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[5] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth & Brooks/Cole.

[6] Thyago P. Carvalho, Fabrízzio A. A. M. N. Soares, Roberto Vita, Roberto da Piedade Francisco, João Pedro Tavares Vieira Basto, and Symone Gomes Soares Alcalá. 2019. A Systematic Literature Review of Machine Learning Methods Applied to Predictive Maintenance. *Computers & Industrial Engineering* 137 (2019), 106024. doi:10.1016/j.cie.2019.106024

[7] Chentong Chen, Mengyuan Zhong, Ye Fan, Jialong Shi, and Jianyong Sun. 2026. HiFo-Prompt: Prompting with Hindsight and Foresight for LLM-based Automatic Heuristic Design. In *International Conference on Learning Representations*. https://openreview.net/forum?id=imSLzfZ6av

[8] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794. doi:10.1145/2939672.2939785

[9] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. 2017. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems* 29, 8 (2017), 3784–3797. doi:10.1109/TNNLS.2017.2736643

[10] Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. 2025. HSEvo: Elevating Automatic Heuristic Design with Diversity-Driven Harmony Search and Genetic Algorithm Using LLMs. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 25 (2025), 26931–26938. doi:10.1609/aaai.v39i25.34898

[11] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. 2022. LIFT: Language-Interfaced Fine-Tuning for Non-Language Machine Learning Tasks. In *Advances in Neural Information Processing Systems*, Vol. 35. 11763–11784. doi:10.52202/068431-0855

[12] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. http://archive.ics.uci.edu/ml

[13] Pierre Geurts, Damien Ernst, and Louis Wehenkel. 2006. Extremely Randomized Trees. *Machine Learning* 63, 1 (2006), 3–42. doi:10.1007/s10994-006-6226-1

[14] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting Deep Learning Models for Tabular Data. In *Advances in Neural Information Processing Systems*, Vol. 34. 18932–18943. https://proceedings.neurips.cc/paper/2021/hash/9d86d83f925f2149e9edb0ac3b49229c-Abstract.html

[15] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?. In *Advances in Neural Information Processing Systems*, Vol. 35. 507–520. doi:10.52202/068431-0037

[16] Sungwon Han, Jinsung Yoon, Sercan O. Arik, and Tomas Pfister. 2024. Large Language Models Can Automatically Engineer Features for Few-Shot Tabular Learning. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 17454–17479. https://proceedings.mlr.press/v235/han24f.html

[17] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. TabLLM: Few-Shot Classification of Tabular Data with Large Language Models. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 206)*. PMLR, 5549–5581. https://proceedings.mlr.press/v206/hegselmann23a.html

[18] Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-1938

[19] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. 2025. Accurate Predictions on Small Data with a Tabular Foundation Model. *Nature* 637, 8045 (2025), 319–326. doi:10.1038/s41586-024-08328-6

[20] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30. 3146–3154.

[21] Nguyen Viet Tuan Kiet, Nguyen Ba Thinh, Thanh Trung Huynh, and Hieu Pham. 2026. MAPLE: Multi-Agent Prior Learning for Constructing Tree Ensembles. In *Workshop on Multi-Agent Learning and Its Opportunities in the Era of Generative AI*. https://openreview.net/forum?id=bBIA4TVEBG

[22] Myung Jun Kim, Léo Grinsztajn, and Gaël Varoquaux. 2024. CARTE: Pretraining and Transfer for Tabular Learning. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 23843–23866. https://proceedings.mlr.press/v235/kim24d.html

[23] Ricardo Knauer, Mario Koddenbrock, Raphael Wallsberger, Nicholas M. Brisson, Georg N. Duda, Deborah Falla, David W. Evans, and Erik Rodner. 2025. "Oh LLM, I'm Asking Thee, Please Give Me a Decision Tree": Zero-Shot Decision Tree Induction and Embedding with Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*. ACM, 1196–1206. doi:10.1145/3711896.3736818

[24] Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn C. Thomas. 2015. Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring: An Update of Research. *European Journal of Operational Research* 247, 1 (2015), 124–136. doi:10.1016/j.ejor.2015.05.030

[25] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 32201–32223. https://proceedings.mlr.press/v235/liu24bs.html

[26] Tennison Liu, Nicolas Huynh, and Mihaela van der Schaar. 2025. Decision Tree Induction Through LLMs via Semantically-Aware Evolution. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=UyhRtB4hjN

[27] John Mingers. 1989. An Empirical Comparison of Selection Measures for Decision-Tree Induction. *Machine Learning* 3, 4 (1989), 319–342.

[28] Jaehyun Nam, Kyuyoung Kim, Seunghyuk Oh, Jihoon Tack, Jaehyung Kim, and Jinwoo Shin. 2024. Optimized Feature Generation for Tabular Data via LLMs with Decision Tree Reasoning. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-2932

[29] Jaehyun Nam, Woomin Song, Seong Hyeon Park, Jihoon Tack, Sukmin Yun, Jaehyung Kim, Kyu Hwan Oh, and Jinwoo Shin. 2024. Tabular Transfer Learning via Prompting LLMs. In *Conference on Language Modeling*. https://openreview.net/forum?id=bo4pauxnIR

[30] J. Ross Quinlan. 1986. Induction of Decision Trees. *Machine Learning* 1, 1 (1986), 81–106.

[31] J. Ross Quinlan. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

[32] Alvin Rajkomar, Eyal Oren, Kai Chen, Andrew M. Dai, Nissan Hajaj, Michaela Hardt, Peter J. Liu, Xiaobing Liu, Jake Marcus, Mimi Sun, Patrik Sundberg, Hector Yee, Kun Zhang, Yi Zhang, Gerardo Flores, Gavin E. Duggan, Jamie Irvine, Quoc Le, Kurt Litsch, Alexander Mossin, Justin Tansuwan, De Wang, James Wexler, Jimbo Wilson, Dana Ludwig, Samuel L. Volchenboum, Katherine Chou, Michael Pearson, Srinivasan Madabushi, Nigam H. Shah, Atul J. Butte, Michael D. Howell, Claire Cui, Greg S. Corrado, and Jeffrey Dean. 2018. Scalable and Accurate Deep Learning with Electronic Health Records. *npj Digital Medicine* 1, 1 (2018), 18. doi:10.1038/s41746-018-0029-1

[33] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6

[34] Dylan Slack and Sameer Singh. 2023. TABLET: Learning From Instructions For Tabular Data. *CoRR* abs/2304.13188 (2023). doi:10.48550/arXiv.2304.13188

[35] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2013. OpenML: Networked Science in Machine Learning. *SIGKDD Explorations* 15 (2013), 49–60. doi:10.1145/2641190.2641198

[36] Zhihan Yang, Jiaqi Wei, Xiang Zhang, Haoyu Dong, Yiwen Wang, Xiaoke Guo, Pengkun Zhang, Yiwei Xu, and Chenyu You. 2026. FORESTLLM: Large Language Models Make Random Forest Great on Few-shot Tabular Learning. arXiv:2601.11311 [cs.LG] doi:10.48550/arXiv.2601.11311

[37] Hangting Ye, Jinmeng Li, He Zhao, Dandan Guo, and Yi Chang. 2025. LLM Meeting Decision Trees on Tabular Data. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-3938

[38] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems*, Vol. 37. 43571–43608. doi:10.52202/079017-1381

[39] Yifei Yuan, Jiatong Li, Weijia Zhang, Mohammad Aliannejadi, Evangelos Kanoulas, and Renjun Hu. 2025. Summarize-Exemplify-Reflect: Data-driven Insight Distillation Empowers LLMs for Few-shot Tabular Classification. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 12324–12348. doi:10.18653/v1/2025.findings-emnlp.659

[40] Tianping Zhang, Zheyu Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Jian Li. 2023. OpenFE: Automated Feature Generation with Expert-level Performance. arXiv:2211.12507 [cs.LG] https://arxiv.org/abs/2211.12507

[41] Zhi Zheng, Zhuoliang Xie, Zhenkun Wang, and Bryan Hooi. 2025. Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 78338–78373. https://proceedings.mlr.press/v267/zheng25o.html